LOGICAL OPERATORS

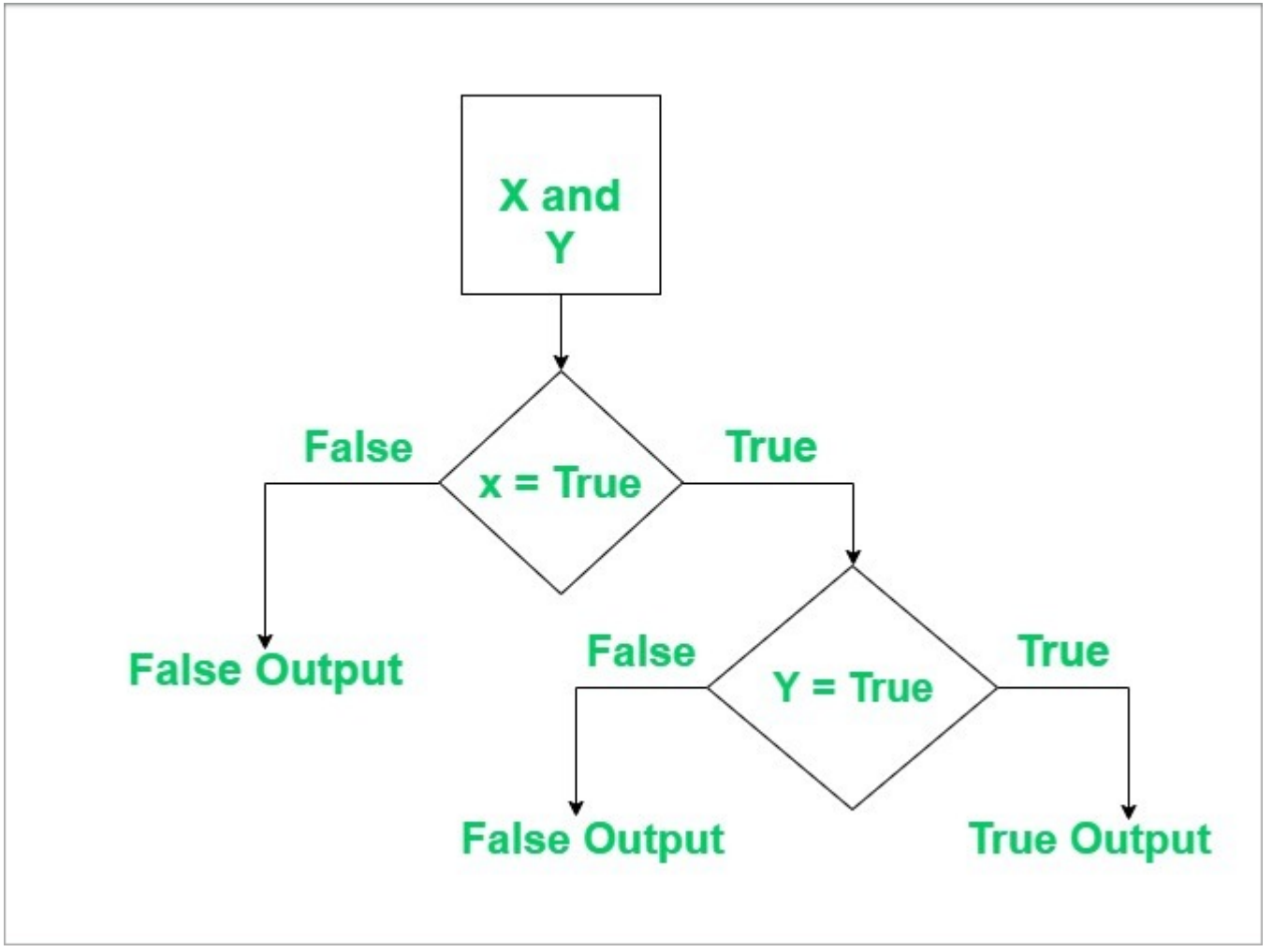
- Python provides three primary logical operators that are essential for creating conditional logic and making decisions.

- Logical operators are also known as boolean operators.

- There are three logical operators in python:
  - and
  - or
  - not

- The operator `and` takes two conditions as arguments and returns `true`,if and only if both conditions are met.
- Given below is a truth table for the operator `and`:

a	b	a and b
T	F	F
F	T	F
F	F	F
T	F	T



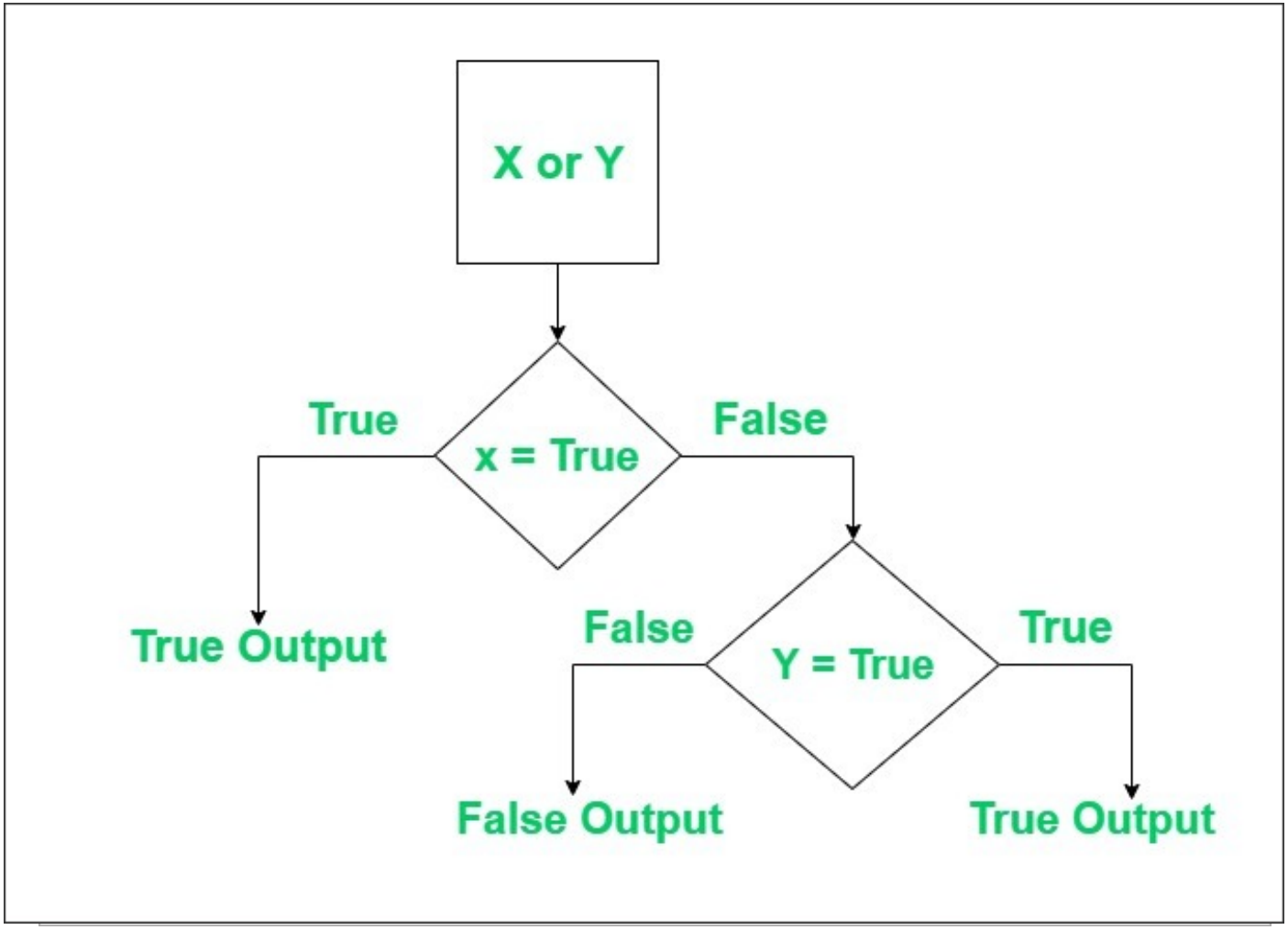
Source: Geeks for geeks

```
a=6
b=4
c=3
d=5
def checkeven(n):
    if n%2==0:
        return True
    else:
        return False
print(checkeven(a) and checkeven(b))
print(checkeven(a) and checkeven(c))
print(checkeven(c) and checkeven(b))
print(checkeven(c) and checkeven(d))
```

Guess the output of the following snippet of code ?

- The operator or takes two conditions as arguments and returns true if either of the both conditions or both of them are met.
- Given below is the truth table for the or operator

a	b	a or b
T	F	T
F	T	T
F	F	F
T	F	T



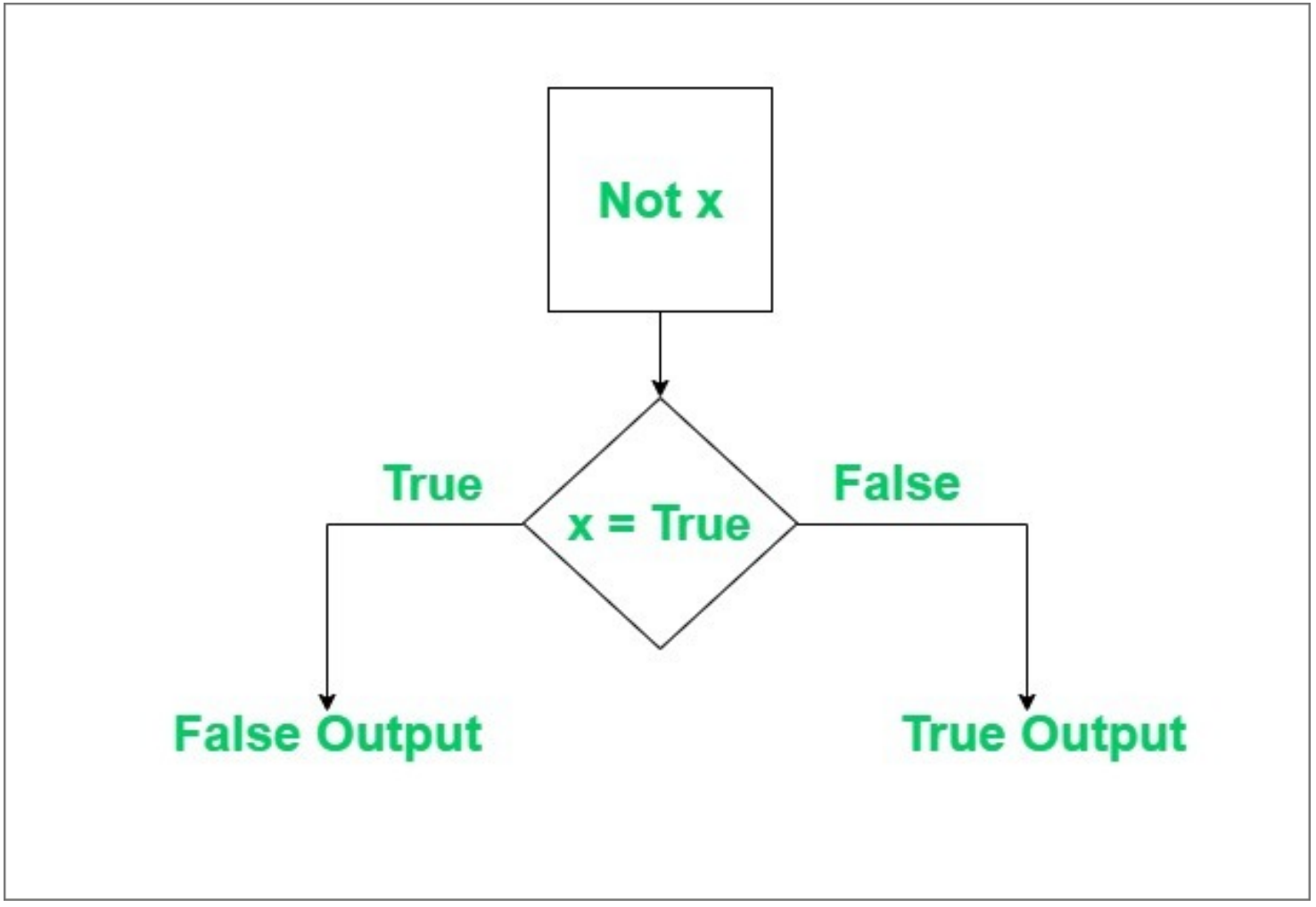
Source: Geeks for geeks

```
a=6
b=4
c=3
d=5
def checkeven(n):
    if n%2==0:
        return True
    else:
        return False
print(checkeven(a) or checkeven(b))
print(checkeven(a) or checkeven(c))
print(checkeven(c) or checkeven(b))
print(checkeven(c) or checkeven(d))
```

Guess the output of the following snippet of code ?

- The operator not takes one condition as an argument and returns true if the condition is false and returns false if the condition is true.
- Given below is the truth table for the not operator

a	b	not(a)	not(b)
T	T	F	F
F	F	T	T



Source: Geeks for geeks

```
a=6

b=4

c=3

d=5

def checkeven(n):

    if n%2==0:

        return True

    else:

        return False

print(not(checkeven(a)))

print(not(checkeven(b)))

print(not(checkeven(c)))

print(not(checkeven(d)))
```

Guess the output of the following snippet of code ?

- All of these operators can be used in conjunction to employ complex conditional logic as a progam demands. The next few slides will depict a few code snippets that utilise all these operators in conjunction.
- While using them together, a close eye must be kept on the placing of the operators since a switch can change much of the output.
- A mental flow-chart of the conditional logic to be applied usually helps in avoiding any errors that may crop up into the code.

```python
username = "user123"

password = "secure456"

remember_me = True


# Using all three logical operators

if len(password) ⩾ 8 and (username ≠ password) and not remember_me:

    print("Strong password and secure settings")

elif len(password) ⩾ 8 or (username ≠ password):

    print("Partially secure setup")

else:

    print("Please improve security")
```

In this code snippet, both **and** and **not** are used in conjunction. While the same can be achieved by also using nested loops, the given method ensures cleaner code.

```python
temperature = 72
is_sunny = True
is_weekend = False


if temperature > 80 and is_sunny:
    print("Great day for swimming!")
elif (temperature > 65 and temperature ⩽ 80) or (is_sunny
and is_weekend):
    print("Good day for hiking!")
elif temperature < 50 or not is_sunny:
    print("Maybe stay indoors today")
else:
    print("Weather is moderate")
```

In this code snippet, **and**, **or**, and **not** are used in conjunction. Again, logical operators often ensure that the codebase is very clean.